

Guided Research: Supporting Formulas in Collaborative Spreadsheets

Korhan Deniz AKIN

Contents

Guided Research: Supporting Formulas in Collaborative Spreadsheets	1
1 General Information	2
2 Representing Spreadsheet Data in Yjs	2
2.1 YColumns & YRows (SpreadSheet.tsx, Cell.tsx)	2
.....	3
.....	3
2.2 YMap (Cell.tsx)	4
2.3 YColKeep & YRowKeep (SpreadSheet.tsx)	6
3 Formula Support	7
3.1 Information	7
3.2 Implementation Details	7
3.2.1 Formula Parser	7
3.2.2 Spreadsheet Store	8
3.2.3 SpreadSheet Component	9
3.2.4 Cell Component.....	9
3.3 Formula Recalculation	9
3.4 Circular Reference Handling.....	10
3.5 What were the issues of the formula support implementation?.....	11
3.6 Benchmarking	11
4 CSV Support	11
5 References	12

1 General Information

2 Representing Spreadsheet Data in Yjs

There are 8 Yjs structures. Yjs is a Javascript framework which uses Conflict-free Replicated Data Types internally to synchronize data across different users. It provides deterministic collaborative environment by abstracting all the low level network messages, merging updates and even websocket connections.

My implementation uses 2 arrays and a map. These arrays store cell IDs and are crucial to display them on the table with respect to the order. The map is used to store the content of the cells. This means there is no global grid, which might have performance consequences during adding/removing columns/rows.

2.1 YColumns & YRows (SpreadSheet.tsx)

These are CRDT-based arrays that are responsible for ordering cells in the spreadsheet. Using a map structure is not enough to ensure ordering, because maps are unordered.

They appear in the SpreadSheet.tsx component. In this component, YColumns and YRows are used to transfer cell IDs to the Spreadsheet's React State when this component is mounted. Figure 2.1 and Figure 2.2 demonstrate them. Later, `columns` and `rows` states are used in the table to display the content.

```

useEffect(() => {
  const observer = (event: any) => {
    event.changes.delta.forEach(
      (change: { insert: string; delete?: any }, key: any) => {
        if (change.insert !== undefined) {
          for (let j: number = 0; j < change.insert.length; j++) {
            let index: number = findYArrayElement(change.insert[j], yColumns);
            setColumns((columns) => [
              ...columns.slice(0, index),
              { id: change.insert[j], positionIndex: index - 1 },
              ...columns.slice(index),
            ]);
            increaseColumnPositionIndexes(index);
          }
        } else if (change.delete !== undefined) {
        }
      },
    );
    const deletedIds = new Set<string>();

    event.changes.deleted.forEach((item: any) => {
      const deletedValues = item.content.getContent() as string[];
      deletedValues.forEach((id) => deletedIds.add(id));
    });

    setColumns((prev) =>
      prev
        .filter((c) => !deletedIds.has(c.id))
        .map((c, i) => ({ ...c, positionIndex: i })),
    );
  };

  yColumns.observe(observer);

  return () => {
    yColumns.unobserve(observer);
  };
}, [yColumns, columns]);

```

Figure 2.1 YColumn observer in SpreadSheet.tsx

```

useEffect(() => {
  const observer = (event: any) => {
    event.changes.delta.forEach(
      (change: { insert: string; delete?: any }, key: any) => {
        if (change.insert !== undefined) {
          for (let i = 0; i < change.insert.length; i++) {
            let index: number = findYArrayElement(change.insert[i], yRows);
            setRows((rows) => [
              ...rows.slice(0, index),
              {
                id: change.insert[i],
                positionIndex: index - 1,
              },
              ...rows.slice(index),
            ]);
            increaseRowPositionIndexes(index);
          }
        }
      },
    );
    const deletedIds = new Set<string>();

    event.changes.deleted.forEach((item: any) => {
      const deletedValues = item.content.getContent() as string[];
      deletedValues.forEach((id) => deletedIds.add(id));
    });

    setRows((prev) =>
      prev
        .filter((r) => !deletedIds.has(r.id))
        .map((r, i) => ({ ...r, positionIndex: i })),
    );
  };

  yRows.observe(observer);

  return () => {
    yRows.unobserve(observer);
  };
}, [yRows, rows]);

```

Figure 2.2 YRow observer in Spreadsheet.tsx

2.2 YMap (Cell.tsx)

To keep values of each cell, YMap was used. It's a CRDT-based key-value pair where ids are defined as *let cellId: string = `\${col.id},\${row.id}`*; and contents are just texts. The catch with the YMap is that for every Cell on the spreadsheet, an observer is defined. This observer is responsible for keeping track of the remote updates of the Cell's own content. This comes with a cost, but the benefits are not negligible.

First of all, with this approach, storing Cell components in a global grid was not needed. If that was the case, adding or removing rows/columns would be costly because of shifting all the remaining elements.

Figure 2.3 illustrates the YMap observer. This observer also has an implementation for Update-Wins CRDT. The idea behind it is to store updates that are concurrent to each other (e.g., offline clients performing updates) and merge them by concatenation. That's the

reason why YMap entries have arrays as their values. Later `appendConcurrentUpdates()` is called to do string concatenation (merging concurrent updates).

```
useEffect(() => {
  const observer = (yMapEvent: any) => {
    yMapEvent.changes.keys.forEach(
      (change: { action: string; oldValue: any }, key: string) => {
        if (change.action === "update" && cellId === key) {
          if (
            yMap.get(cellId) !== undefined &&
            yMap!.get(cellId)!.length === 0
          ) {
            return;
          }
          let length: number = yMap!.get(cellId)!.length;
          for (let i = 0; i < length; i++) {
            if (
              change.oldValue[0] !== undefined &&
              change.oldValue[0].id !== yMap!.get(cellId)![i].id
            ) {
              yMap!.get(cellId)!.push(change.oldValue[0]);
            }
          }
          if (yMap!.get(cellId)!.length === 1) {
            //if there are no concurrent operations
            setContent(yMap!.get(cellId)![0].content);
            return;
          }
          let currentCell = getCellContent(key)![0];
          let cellFinalContent: string = appendConcurrentUpdates();
          setYMapContent(
            cellId,
            yMap.get(cellId)![0].id,
            cellFinalContent,
            currentCell.formula ?? undefined,
            currentCell.formulaReferenceCellIds ?? undefined,
          );
          console.log("formula: ", currentCell.formula);
        } else if (change.action === "update" && cellId !== key) {
        } else if (
          change.action === "add" &&
          cellId === key &&
          yMap.get(key) !== undefined &&
          yMap!.get(key)!.length === 1
        ) {
          setContent(yMap!.get(key)![0].content);
        }
      },
    );
  };
  yMap.observe(observer);
  return () => {
    yMap.unobserve(observer);
  };
}, [useYMapStore.getState()]);
```

Figure 2.3

2.3 YColKeep & YRowKeep (SpreadSheet.tsx)

With standard Yjs, data was lost when a user modified a cell and, at the same time, another user deleted the corresponding column or row. Spreadsheet addresses such data loss by using “keep” arrays for columns and rows [1]. When there is a “positive update” on a cell:

“keep id” in Figure 2.4 is stored in those arrays. Clients monitor the changes of arrays with observers, and if they are non-empty, “undo” operations are performed equal to the number of elements in the array structures. Figure 2.5 demonstrates the observer implementation for YRowKeep and YColKeep arrays.

```
let keepId: RemoveKeepOperationId = `c${YDoc.clientID as number}.${1 as number}`;  
yColKeep.set(col.id, [keepId]);  
yRowKeep.set(row.id, [keepId]);
```

Figure 2.4 Keep ID

```
useEffect(() => {  
  yColKeep.observe((event: any) => {  
    if (event.transaction.origin) {  
      yColKeep.forEach((_: any, key: string) => {  
        if (yColumns.toArray().indexOf(key) < 0) {  
          undoColumns.undo();  
          undoMap.undo();  
        }  
      });  
      yColKeep.clear();  
    }  
  });  
  
  yRowKeep.observe((event: any) => {  
    if (event.transaction.origin) {  
      yRowKeep.forEach((_: any, key: string) => {  
        if (yRows.toArray().indexOf(key) < 0) {  
          undoRows.undo();  
          undoMap.undo();  
        }  
      });  
      yRowKeep.clear();  
    }  
  });  
  
  undoColumns.on("stack-item-added", () => {  
    undoColumns.undoStack.forEach((item: any, index: any) => {  
      if (item.insertions.clients.size > 0)  
        undoColumns.undoStack.splice(index, 1);  
    });  
  });  
  
  undoRows.on("stack-item-added", () => {  
    undoRows.undoStack.forEach((item: any, index: any) => {  
      if (item.insertions.clients.size > 0)  
        undoRows.undoStack.splice(index, 1);  
    });  
  });  
}, []);
```

Figure 2.5 YColKeep & YRowKeep implementation with UndoColumns and UndoRows in Spreadsheet.tsx

3 Formula Support

3.1 Information

We thought support of formulas in a collaborative environment would be interesting for several reasons:

- a. The implementation itself.
- b. Secondly, retrieving spreadsheet data for formula calculation. What is the feasible way of doing this without ruining the spreadsheet if there are offline users too?
- c. Thirdly, formula recalculation for every update and its probable bottlenecks. For example, let's say a result is computed for a formula using 1000 cells and a user updates one cell value by incrementing it by one. In this scenario, should the formula be calculated from the very beginning, or is it possible to support this functionality more efficiently? While the current implementation does so by recalculating the formula, hard coding improvements by addressing supported formulas is possible.
- d. What happens when a new row or column is added in between formula reference cells?
- e. Lastly, addressing probable issues such as circular referencing during calculation and recalculation.

3.2 Implementation Details

In this section, I will explain the related parts of functions/components for formula calculation.

3.2.1 Formula Parser

First, the `'fast-formula-parser'` [2] library has been used for formula functionality. The main implementation for this library is in `'formula.ts'`. In this file, there are 2 functions: `'parser()'`, which comes from the library itself, and `'parserDriver()'` which calls the `'parser()'` with `'spreadsheetData'` obtained in the Spreadsheet component.

This `parserDriver()` function returns the formula result and returns the IDs of the cells that have been marked (or registered) while computing the formula result. Figure 3.1 demonstrates this function.

```
const parserDriver = (
  spreadsheetData: string[][],
  formula: string,
  position,
) => {
  data = spreadsheetData;
  markedCells = [];
  let rowLength: number = data.length;
  let collenght: number = data[0].length;
  for (let i = 0; i < collenght; i++) {
    for (let j = 0; j < rowLength; j++) {
      if (!Number.isNaN(Number(data[i][j]))) {
        data[i][j] = Number(data[i][j]);
      } else data[i][j] = "";
    }
  }
  let result = {
    markedCells: markedCells,
    formulaResult: parser.parse(formula, position, true),
  };
  return result;
};
```

Figure 3.1 `parserDriver()` function which calls `parser()` function which is from the `fast-formula-parser` library.

3.2.2 Spreadsheet Store

To retrieve the store globally, Zustand, which is a small, fast, and scalable state management solution [3], has been used. It is similar to Redux but was easier to implement. Normally, the direction for the data transfer is from the parent to the child. However, it would not have been feasible to transfer all spreadsheet data from the Spreadsheet component to every Cell. The latter solution requires *spreadsheet data * num of total cells*. With Zustand, there is a single spreadsheet data to be used for formula calculation.

`spreadSheetData` is the name of the object that contains all values in each cell and is a matrix of strings obtained from `SpreadSheetStore.ts`: `returnSpreadSheet()`. As mentioned in section 3.1 this function returns all spreadsheet data (optimization might be possible).

```
const returnSpreadsheet = (): string[][] => {
  let matrix: string[][] = [];
  for (let i = 0; i < yRows.length; i++) {
    let currentRow: string[] = [];
    for (let j = 0; j < yColumns.length; j++) {
      let cellId: string = `${yColumns.get(j)},${yRows.get(i)}`;
      let cellData = yMap.get(cellId)![0];
      let content = cellData ? cellData.content : "";
      currentRow.push(content);
    }
    matrix.push(currentRow);
  }
  return matrix;
};
```

Figure 3.2 returnSpreadSheet() in SpreadSheetStore.ts

3.2.3 SpreadSheet Component

3.2.3.1 handleFormula()

3.2.4 Cell Component

3.2.4.1 handleFocusOut()

3.3 Formula Recalculation

Formula cells register candidate cells in the `markedCells[]` array when the formula is defined to compute the result. Similarly, candidate cells store IDs of the formula cells in the `formulaReferenceCells[]` array. This is an array because there might be multiple formula cells registering the same cell. These 2 arrays are kept in every Cell component and are defined in `CellType`. As in Figure 4.3 arrays are optional, denoted with "?", so it is not a problem if they are undefined.

```
type CellType = {
  id: string;
  content: string;
  formula?: string;
  formulaReferenceCellIds?: string[];
  markedCells?: string[];
};
```

Figure 3.3 Cell type definition

After these operations, the `handleFocusOut()` function in the Cell component is responsible for the formula recalculation as shown in Figure 4.4.

```
// this conditional is to update the content of the main formula cell. ?. is "optional chaining".
let i = 0;
while (
  currentCell.formulaReferenceCellIds?.[i] !== undefined &&
  currentCell.formulaReferenceCellIds[i] !== cellId
) {
  setContent(e.target.value);
  setYMapContent(
    cellId,
    YDoc.clientID,
    e.target.value,
    undefined,
    currentCell.formulaReferenceCellIds,
    undefined,
  ); //updating the current cell itself

  let referencedCellId: string = currentCell.formulaReferenceCellIds[i];
  let referencedCell = getCellContent(referencedCellId)?.[i];
  formula = referencedCell?.formula;

  let result: { markedCells: string[]; formulaResult: string } =
    handleFormula(
      formula as string,
      {
        row: rowIdx,
        col: colIdx,
        sheetName: "spreadsheet",
      },
      referencedCellId,
    );
  setYMapContent(
    referencedCellId,
    YDoc.clientID,
    result.formulaResult,
    formula,
    undefined,
    referencedCell?.markedCells,
  ); //here updating the referenced formula cell.
  i += 1;
}
```

Figure 3.4 The conditional which is responsible from the formula recalculation inside `handleFocusOut()` function.

3.4 Circular Reference Handling

A circular reference occurs when a formula refers directly or indirectly back to its own cell value, creating an endless calculation loop. For example, if cell A1 contains the formula $=A1+B1$, then A1 references itself [4]. Actually, it is not an issue when defining the formula (the cell is not yet related to the corresponding formula). However, when there are updates to cells marked by formulas, a circular reference becomes apparent. Meaningfully, when there is formula recalculation (as explained in section 3.3).

My implementation does not produce an endless loop because formula recalculation does not use observers. Rather, it is being done via the `handleFocusOut()` function in Cell components. Therefore, a formula is calculated via `handleFocusOut()` calls. Even though there are no calculation infinite loops, when a user wants to increase a cell value by one (if

the cell is marked and the formula is SUM), as in a circular reference case, formula cells register themselves, the existing result from the previous step, so it does not increase by one rather *formula's previous result* + 1. This produces a result that the user does not intend.

To solve this, Microsoft Excel's approach is used. When it detects a circular reference, the formula value is set to 0. In my implementation, I have a very basic approach that can be seen in Figure 4.5. This approach might be improved in terms of time complexity via Depth-First Search.

```
for (let i = 0; i < result.markedCells.length; i++) {  
  if (formulaCellId === result.markedCells[i]) {  
    console.log("Circular reference detected!");  
    result = {  
      markedCells: [],  
      formulaResult: "0",  
    };  
  }  
}  
return result;  
};
```

Figure 3.5 Very basic circular reference detection.

3.5 What were the issues of the formula support implementation?

1. Defining `formulaReferenceCellIds` and `markedCells` as normal arrays rather than Y.Arrays
 - a. These arrays do not get updated in remote clients.

3.6 Benchmarking

4 CSV Support

5 References

- [1] P. B. a. A. B. E. Yanakieva, "A Study of Semantics for CRDT-based Collaborative Spreadsheets," in *Proceedings of the 10th Workshop on Principles and Practice of Consistency for Distributed Data*, Rome, Italy, 2023.
- [2] L. Lyu, "fast-formula-parser," 2020. [Online]. Available: <https://www.npmjs.com/package/fast-formula-parser>. [Accessed 24 July 2026].
- [3] Poimandres, "Introduction - Zustand," Zustand Documentation., [Online]. Available: <https://zustand.docs.pmnd.rs/learn/getting-started/introduction>. [Accessed 25 07 2026].
- [4] Macabacus, "Circular Reference in Excel: How to Find and Fix Errors.," Macabacus, [Online]. Available: <https://macabacus.com/blog/conquering-circular-references-in-excel>. [Accessed 23 July 2026].